The Convergence of Curvature Variable Physics and High-Performance Computing

A Strategic Analysis of 206 Innovation Inc.'s CVP Spin Overlay and NVIDIA Inception Integration

c_eff Canonical Equation Edition | Updated Equation Canon + Dillon Cryptography Integration

Timothy J. Dillon | 206 Innovation Inc. | Bellevue, Washington | 2026

$$c_{eff} = f(K, R, \nabla K, \partial_t K)$$

Effective propagation is curvature-conditioned; local c remains invariant.

$$D(c_{eff}, h, \mathcal{G}) = (\delta c_{eff} / \delta \mathcal{G})|_h$$

Formal Dillon Operator over metric/gravitational geometry at fixed informational scale h.

Executive Abstract

This updated edition replaces legacy variable-c language with the locked c_eff canonical notation. The revision preserves the local relativistic invariant c while defining curvature-conditioned effective propagation as c_eff, the engineering and field-theoretic variable used by the CVP Spin Overlay (CSO). The update is important for NVIDIA-oriented high-performance computing because it clarifies that the overlay does not claim to change local causality; rather, it uses geometry-conditioned propagation, tensor-state invariants, scheduling metadata, and proof-bundle replay to govern computational integrity.

The revised CSO is positioned as a GPU-resident integrity plane for deterministic replay, stale-state suppression, tensor contradiction detection, and high-assurance auditability. It complements lower-level reliability mechanisms by adding algorithm-aware curvature invariants, TEVR-aligned proof bundles, and a policy-controlled trust plane that can be integrated with CUDA, NCCL, GPUDirect Storage, NVLink, InfiniBand, and AI-factory workflows.

This edition also adds a Dillon Cryptography extension. The cryptographic interpretation treats binary strings, tensor states, and execution traces as structural objects whose exposure, reducibility, symmetry leakage, closure risk, and self-healing readiness can be scored. The result is a unified architecture in which physics-aware computing and cryptographic self-healing are governed by the same curvature-conditioned canon.

1. Canonical Equation Control Update

The original paper described the Dillon Equation as a direct dependence of the propagation constant c on scalar curvature R. This updated version corrects that framing. The local relativistic speed c remains the invariant causal boundary condition measured in local inertial frames. The variable that changes across

structured geometry is c_eff: the effective propagation behavior through metric, curvature, gradient, and temporal curvature conditions.

Legacy wording to replace	Canonical replacement	Reason
c = f(R)	c_eff = f(R)	Use only where the document refers to effective propagation through regional curvature, not local c.
D(c, ħ, R) = ∂c/∂R	D(c_eff, h, $\mathscr{G}$) = (δc_eff / δ$\mathscr{G}$)|$_h$	Formal manuscripts should use the functional derivative over metric/gravitational geometry.
Variable speed of light	Curvature-conditioned effective propagation	Avoids implying local Lorentz invariance is being rejected.
Propagation limit of information and gravity (c)	Effective propagation behavior of information and gravity (c_eff)	Separates local causal speed from global/geometric propagation behavior.
Gcal or G when used for geometry	$\mathscr{G}$	Prevents confusion with Newtonian G and denotes the full metric-curvature state.

1.1 Locked equation family

$$c = \text{local invariant causal speed}$$

$$c\_eff = f(K, R, \nabla K, \partial_t K)$$

$$D(c\_eff, h, \mathscr{G}) = (\delta c\_eff / \delta \mathscr{G})|_h$$

$$T\_\mu\nu \rightarrow \mathscr{G} \rightarrow c\_eff$$

$$\alpha\_D(\mathscr{G}, K) = e^2 / (4\pi \varepsilon_0 \hbar\, c\_eff)$$

The short-form public equation c_eff = f(R) remains acceptable for graphics and executive exposition. Formal papers, patents, SDK specifications, and validation protocols should use the full expression c_eff = f(K, R, ∇K, ∂$_t$K) and the functional Dillon Operator over $\mathscr{G}$.

1.2 Expanded definition of gravitational geometry

$$\mathscr{G} = (g\_\mu\nu, R\_\mu\nu, R, K, \nabla K, \partial_t K, \Gamma\hat{}\lambda\_\{\mu\nu\})$$

Symbol	Canonical meaning	Role in CSO/CVP
g_μν	Metric tensor	Defines spacetime interval, local relativistic geometry, and the computational analogy for bounded execution geometry.
R_μν	Ricci curvature tensor	Encodes curvature sourced by stress-energy; in the CSO analogy,

		supports tensor-field curvature mapping.
R	Ricci scalar	Supports short-form c_eff = f(R) expressions and regional curvature summaries.
K	Curvature field/state	Primary CVP state variable for structured propagation and admissible execution trajectories.
∇K	Spatial curvature gradient	Captures regional variation, path dependence, and drift vectors across distributed systems.
$\partial_t K$	Temporal curvature evolution	Captures changing geometry and time-dependent execution-state drift.
Γ^λ_{μν}	Connection coefficients	Describe how propagation direction/state changes across the manifold or execution graph.

2. Strategic Context: 206 Innovation Inc. and NVIDIA Inception

The emergence of 206 Innovation Inc. as a participant in the NVIDIA Inception ecosystem marks a transition from theoretical CVP architecture toward an implementation pathway for deterministic high-performance computing. The CSO is framed as a software-first integrity layer that can mature from developer libraries and runtime wrappers into a policy-controlled service plane for high-assurance GPU workloads.

The business and technical relevance of the Inception pathway is not merely branding. For a deep-technology system, access to developer training, GPU cloud resources, preferred hardware/software pricing, solution engineering support, investor visibility, and go-to-market networks can compress the validation cycle from theoretical framework to pilot-grade reference implementation.

Inception benefit category	Value for 206 Innovation Inc.	Impact on CSO development
Technical resources	DLI training, developer resources, SDK exposure	Improves CUDA, kernel-instrumentation, and profiling competence.
Cloud/GPU access	GPU-backed simulation and benchmarking environments	Enables reproducibility testing, fault injection, and large-scale EM/digital-twin experiments.
Preferred pricing	Discounted hardware/software pathways	Supports on-premise reference systems and customer pilots.
Go-to-market support	Startup visibility, ecosystem introductions, conference access	Creates routes into defense, telecom, public sector, and AI-factory buyers.
Engineering guidance	Architecture-level review and optimization support	Helps align CSO design with Hopper, Blackwell, Rubin, and next-generation GPU workflows.

3. Revised Theoretical Foundation: From Variable c to c_eff

The central paradigm shift is the distinction between local causality and effective propagation. In the revised CVP notation, the CSO does not rely on a claim that the local speed of light is altered. Instead, it treats the propagation of information, data-state transitions, and tensor evolution as geometry-conditioned processes. This is the correct bridge from physical CVP theory into GPU computing: local hardware operations remain bounded by conventional device physics, while system-level propagation, execution ordering, memory visibility, and distributed state coherence are governed through an effective geometry model.

3.1 Curvature-conditioned computation

Within a GPU cluster, computational curvature is an abstracted geometry over tensors, kernels, scheduling order, memory state, communication boundaries, and storage provenance. The CSO maps those objects into a state manifold and checks whether the computation remains on an admissible trajectory. The updated c_eff notation is therefore useful because it is an effective propagation variable, not a replacement for physical c.

$$c\_eff^{GPU} = f(K\_tensor, R\_exec, \nabla K\_fabric, \partial_t K\_state)$$

GPU execution analogy: tensor curvature, execution curvature, fabric gradients, and state-time evolution.

3.2 Harmonic gating and deterministic control epochs

The 3-6-9 schedule is retained, but it is now expressed as a control envelope over c_eff rather than over c. It defines deterministic windows for sampling, curvature estimation, state gating, actuation, and proof generation.

$$G_{369}(t) = \Sigma\_\{j\in\{3,6,9\}\} a\_j \sin(2\pi jt/T + \varphi\_j)$$

$$\theta\_t = \Pi\_window(G_{369}(t), K\_t, \partial_t K\_t, c\_eff, t)$$

θ_t is the admissible control epoch for state update, replay checkpoint, or proof-bundle seal.

The practical interpretation is straightforward: GPU kernels and distributed collectives are sampled or sealed at deterministic epochs. These epochs reduce audit ambiguity by anchoring when state was observed, which invariant was checked, and what replay metadata must be preserved.

4. The CVP Spin Overlay as a GPU-Resident Integrity Plane

The CSO is a lightweight, opt-in integrity plane that operates across GPU compute, memory, storage, and interconnect fabric. It complements existing RAS and ECC mechanisms by adding algorithm-aware invariant checks. ECC can catch many bit-level problems; the CSO is designed to detect contradictions in the mathematical and structural behavior of a workload even when each individual bit or tensor value appears plausible.

- Silent data corruption: identify plausible but structurally inconsistent outputs.

- Nondeterminism at scale: bound run-to-run drift from reductions, atomics, race conditions, asynchronous overlap, and architecture-specific rounding.
- Stale or inconsistent state: detect outdated tensors or conflicting staged reads/writes across overlapping streams.
- Audit gaps: create replayable proof artifacts that capture seeds, kernel versions, state hashes, scheduling metadata, and invariant checkpoints.

4.1 Invariant verification loop

$$I_t = H(state_t \| kernel\_id \| seed \| \theta_t \| c_{eff,t} \| policy\_id)$$

Hash-anchored invariant checkpoint for each selected control epoch.

$$\Delta I_t = dist(I_t, I_t^{replay}) \leq \varepsilon_{policy}$$

Replay succeeds when the live/replay deviation remains inside policy tolerance.

The invariant loop does not require every operation to be serialized. A production implementation can sample high-risk regions, seal boundary events, and use configurable policy thresholds so overhead remains acceptable for high-throughput AI and simulation workloads.

4.2 Curvature contradiction score

$$C_{CSO}(t) = w_1 R_{red}(t) + w_2 S_{leak}(t) + w_3 L_{closure}(t) + w_4 D_{drift}(t) - w_5 H_{heal}(t)$$

C_CSO(t) is the runtime contradiction score. R_red captures reducibility or repeated structural collapse, S_leak captures symmetry leakage, L_closure captures closure or stale-state risk, D_drift captures replay or distributed divergence, and H_heal captures available self-healing margin. A high score triggers alert, containment, replay, or state regeneration depending on policy.

5. NVIDIA Architecture Integration

Layer	Integration mechanism	CSO purpose
CUDA kernels	Kernel wrappers, macros, tensor-view probes, optional instrumentation	Capture invariant checkpoints without disrupting primary Tensor Core paths.
NCCL collectives	Boundary tags, collective-step sealing, distributed state comparisons	Detect stale or inconsistent distributed tensor propagation.
NVLink / InfiniBand / Ethernet	Fabric telemetry correlation	Accelerate root-cause isolation for cluster-scale anomalies.
GPUDirect Storage	End-to-end provenance tags from object/NVMe to GPU memory	Reduce CPU-staging ambiguity and protect async pipelines.
Morpheus / cybersecurity stack	Security telemetry integration and anomaly-policy routing	Extend CSO events into cyber defense workflows.
Omniverse / digital twin	Simulation-grounded fault injection and resilience visualization	Validate EM, HEMP, EW, and directed-energy scenarios before field deployment.

6. TEVR Proof Bundles and Audit-Ready Replay

The Proof Bundle is the principal audit artifact of the CSO. It is designed to be Transparent, Encrypted, Verifiable, and Reproducible. A bundle should allow an independent reviewer or partner lab to rerun a computation, inspect the policy envelope, reproduce the same results within tolerance, and identify where any divergence began.

Proof Bundle field	Description
Workload manifest	Model, simulation, dataset, code branch, container, driver, and dependency fingerprint.
Kernel/version anchors	Kernel identifiers, compiler flags, library versions, architecture target, and runtime settings.
Seeds and scheduling metadata	Random seeds, stream/graph scheduling metadata, control epochs, and deterministic replay policy.
Invariant hashes	I_t checkpoints, tensor-boundary hashes, storage provenance hashes, and NCCL boundary tags.
c_eff policy envelope	Curvature-state variables, gating schedule, tolerance epsilon, and contradiction thresholds.
Incident trail	Detected anomalies, containment actions, replay attempts, and self-healing decisions.

$$\text{TEVR\_score} = N_{\text{successful\_replays}} / N_{\text{total\_replay\_attempts}}$$

For defense and federal-lab pilots, the TEVR score becomes a practical acceptance metric. It converts a broad claim of reproducibility into a measured percentage of successful bit-for-bit or policy-tolerant replays across hardware, node counts, and perturbation conditions.

7. Dillon Cryptography Extension

This update adds the Dillon Cryptography layer to the NVIDIA paper. The cryptographic extension treats binary sequences, tensors, model weights, execution traces, and communications boundaries as structural states. Instead of analyzing only bit entropy or key length, the framework also scores curvature morphology: reducibility, symmetry leakage, closure risk, drift, and self-healing readiness.

7.1 Binary Curvature Decoding Engine

Primitive	Structural interpretation
0	Boundary, damping, latent basin, compression, null interval.
1	Activation, curvature rise, impulse, asserted state.
00	Deep suppression or stabilized basin.
11	Reinforced activation or dual peak.
01	Emergence from compression.
10	Closure after activation.
010	Isolated pulse bounded on both sides.
111+	Strong propagation pressure or possible divergence front.

7.2 Exposure and self-healing score

$$C(K) = \lambda_1 R\_red(K) + \lambda_2 S\_sym(K) + \lambda_3 X\_exp(K) + \lambda_4 E\_entropy(K) - \lambda_5 H\_heal(K)$$

C(K) is both diagnostic and prescriptive. It flags whether a key, tensor boundary, trace, packet, or session is structurally exposed, while also identifying whether the system has enough self-healing margin to regenerate, rebound, or migrate the state.

Component	Meaning	Operational response
R_red(K)	Reducibility or collapse-friendly structure	Raise migration urgency or reject weak state.
S_sym(K)	Symmetry leakage or repeated topology	Trigger rekeying, reshaping, or additional entropy injection.
X_exp(K)	Exposure level under structural attack	Escalate to containment or high-assurance proof capture.
E_entropy(K)	Entropy distribution and activation pressure	Rebalance sampling or regenerate state.
H_heal(K)	Self-healing capacity	Allow rebound, recovery, or Omega-session regeneration.

7.3 Cryptographic primitives aligned to CSO

Primitive / component	Purpose in Dillon Cryptography	CSO integration point
Dillon-KX	Shared-secret establishment with rebound-capable session regeneration.	Used for GPU-cluster session sealing and policy-bound proof exchange.
Dillon-SIG	Authentication/signing with structural admissibility checks.	Signs Proof Bundles and validates workload provenance.
Dillon-HASH	Integrity/compression primitive designed to disrupt collapse-friendly round structure.	Supports invariant hashes and replay checkpoints.
Omega-SESSION	Live trust-state classification and self-healing triggers.	Routes anomaly state into alert, containment, replay, or regeneration.
Exposure Engine	Scores reducibility, closure risk, symmetry leakage, migration urgency, and healing readiness.	Feeds C_CSO(t), TEVR policy, and cyber telemetry.

8. Strategic Application Domains

Domain	Primary pain point	CSO value proposition
Defense / national security	EM/HEMP/EW/directed-energy resilience; auditability for simulations and C2/C4I decisions.	Deterministic execution traces, fault isolation, proof bundles, and survivability modeling.
Telecommunications / 6G-7G	Physical-layer trust, distributed timing, and adaptive resonance	Curvature-conditioned timing models and authenticated

	scheduling.	propagation signatures.
AI factories / LLM training	Run-to-run drift, silent numerical anomalies, irreproducible training failures.	Replayable training windows, tensor-boundary invariants, reduced wasted GPU weeks.
CFD / molecular dynamics	Error accumulation over millions/billions of particle or cell updates.	Curvature-aware correction windows and policy-tolerant replay.
Federal labs / science	Need for reproducible, independently verifiable computational results.	TEVR evidence package and third-party replay harness.
Cybersecurity / cryptography	Structural exposure beyond conventional entropy metrics.	BCDE, C(K), Omega-session governance, and self-healing trust response.

9. Implementation Roadmap

Phase	Duration	Primary deliverable	Acceptance gate
Phase 0: Baseline & Harness	2-4 weeks	Reference nondeterminism/anomaly baseline; replay harness; workload selection.	Baseline drift, rerun cost, and silent anomaly rate measured.
Phase 1: Determinism & Proof Bundles	4-8 weeks	Kernel/version anchoring, policy envelope, initial Proof Bundle schema.	Third-party replay succeeds within defined tolerance.
Phase 2: Multi-GPU & Storage	8-12 weeks	NCCL boundary tags, GDS provenance tags, distributed anomaly capture.	Stale-state and distributed contradiction tests detected.
Phase 3: Cluster Integrity Ops	12+ weeks	Telemetry correlation, fault injection, incident report generator.	TEVR score and time-to-root-cause improve against baseline.
Phase 4: Partner Pilot	M12-M18	Defense/telecom/AI-factory pilot with policy-controlled deployment.	Partner acceptance package and reproducibility report.
Phase 5: SDK Packaging	M18+	C/C++/Python SDK, conformance suite, partner integration guide.	Developer-ready package and documented operational envelope.

10. KPI Framework

KPI category	Metric	Target interpretation
Integrity	Contradiction rate	Reduction in undetected structurally inconsistent outputs.
Integrity	Stale-state suppression	Percentage of stale tensor or pipeline events detected and contained.
Replay	TEVR score	Successful bit-for-bit or policy-

		tolerant replay rate across nodes/architectures.
Operations	Time-to-root-cause	Time required to isolate whether failure came from kernel, fabric, storage, or policy state.
Efficiency	Rerun avoidance	Reduction in wasted GPU cycles/weeks caused by irreproducible failures.
Security	Exposure score reduction	Improvement in C(K) or C_CSO(t) after self-healing, regeneration, or migration.
Performance	Overlay overhead	Configurable cost of instrumentation under sampled, boundary-only, or high-assurance modes.

11. Updated Conclusion

The c_eff canonical update strengthens the NVIDIA Inception paper by making the framework more precise, defensible, and implementation-ready. The revised formulation preserves local relativistic invariance while providing a formal effective propagation variable for geometry-conditioned computing. This resolves the most important notation risk in the original paper: conflating local c with system-level propagation behavior.

With the revised equation family, the CVP Spin Overlay becomes easier to position as a high-assurance GPU integrity plane. Its claim is not that GPUs alter fundamental causality; its claim is that distributed computation can be governed through effective propagation geometry, invariant checkpoints, harmonic control epochs, TEVR proof bundles, and cryptographic self-healing. That framing aligns the paper with NVIDIA-scale deployment realities while preserving the distinctive Dillon Architecture thesis.

The next development priority is therefore clear: translate the canonical equations into a reference SDK specification, define benchmark workloads, implement the invariant hashing and replay harness, and use the TEVR score as the measurable bridge from theory to partner validation.

Appendix A - Canonical Replacement Checklist

Document section	Update applied
The Dillon Equation and Variable Speed of Light	Retitled and reframed as c_eff canonical effective propagation; local c preserved.
3-6-9 Master Equation	Updated to a deterministic control epoch over c_eff and state curvature.
Curvature-dependent integrity checks	Mapped to I_t, ΔI_t, C_CSO(t), and TEVR replay tolerance.
Proof Bundle section	Expanded with workload manifest, kernel anchors, policy envelope, invariant hashes, and incident trail.
Cryptography section	Added BCDE, C(K), Dillon-KX, Dillon-SIG, Dillon-HASH, Omega-SESSION, and Exposure Engine.
Conclusion	Updated to emphasize local invariance, effective propagation, deterministic replay, and partner validation.

Appendix B - Source Basis

- Original source paper: NVIDIA Inception Program Deep Dive, uploaded by Timothy J. Dillon.
- Canonical equation basis: Dillon Equation Core Canon v2 / Canonical c_eff Release v2.
- Notation update basis: Dillon Equation Canonical Notation Update Pack v2 FINAL.
- Architecture basis: Dillon Architecture v9 locked equation canon and Dillon Architecture full manuscript stack.
- Cryptography basis: Dillon Framework Accomplishment Brief, Dillon Cryptography Solution, BCDE/Omega-session materials, and the Dillon Cryptography platform primitives.